PyTorch keeps track of all forward computations and stores their derivatives so that eventually when we call the `backward()` method on the output node of our computational graph, it will backpropagate through these gradient functions node by node until the input node. That's how we get the gradients for all the parameters in the model.

Notice that we also called the `detach()` method on the `Y` tensor. This was actually unnecessary, since we used `torch.no_grad()` when we computed `newQ`, but we included it because detaching nodes from the computational graph will become ubiquitous throughout the rest of the book, and not properly detaching nodes is a common source of bugs when training a model. If we call `loss.backward(X,Y)`, and `Y` was associated with its own computational graph with trainable parameters, we would backpropagate into `Y` *and* `X,` and the training procedure would learn to minimize the loss by updating the trainable parameters in the `X` graph and the `Y` graph, whereas we only want to update the `X` graph. We *detach* the `Y` node from the graph so that it is just used as data and not as a computational graph node. You don't need to think too hard about the details, but you do need to pay attention to which parts of the graph you're actually backpropagating into and make sure you're not backpropagating into the wrong nodes.

You can go ahead and run the training loop—1,000 epochs will be more than enough. Once it's done, you can plot the losses to see if the training is successful and the model converges. The loss should more or less decrease and plateau over the training time. Our plot is shown in figure 3.10.

Figure 3.10. The loss plot for our first Q-learning algorithm, which is clearly down-trending over the training epochs.